

Assignment 10

Author: Lan Stare Date: 25. 11. 2025

(a)

Remark: We chose the coefficient of $t \cdot p_{n-1}$ to be 1, so we don't have to prove that $\delta_n = 1$.

NM Assignment 10

$S_n := 0 \ ; \ n \geq 1$
 $\gamma_n = \frac{n-1}{\sqrt{4(n-1)^2 - 1}} \ ; \ n \geq 1$

Inserting S_n and γ_n : $p_n(t) = t \cdot p_{n-1}(t) - \frac{(n-1)^2 p_{n-2}(t)}{4(n-1)^2 - 1}$

base cases :

- $L_0(t) = p_0(t) = 1$
- $p_1(t) = (t - 0) - \left(\frac{1-1}{0-1}\right)^2 = t = L_1(t)$

$\rightarrow L_n(t) := \frac{2n-1}{n} t \cdot L_{n-1}(t) - \frac{n-1}{n} L_{n-2}(t)$

$p_0(t) = 1, \ p_1(t) = (t - S_1) - \gamma_1^2$ $n \geq 2$
 $p_n(t) = (t - S_n) p_{n-1}(t) - \gamma_n^2 p_{n-2}(t)$

DUE TO DIFFERENT NORMALIZATION, $p_n(t) = d_n \cdot L_n(t)$

$$p_n(t) = d_n \cdot L_n(t) = d_n \cdot \frac{2n-1}{n} t \cdot L_{n-1}(t) - d_n \cdot \frac{n-1}{n} L_{n-2}(t) =$$
$$= d_n \cdot \frac{2n-1}{n} t \cdot \frac{1}{d_{n-1}} \cdot p_{n-1}(t) - d_n \cdot \frac{n-1}{n} \cdot \frac{1}{d_{n-2}} \cdot p_{n-2}(t)$$

$\frac{d_n}{d_{n-1}} \left(\frac{2n-1}{n} \right) = 1$

$\frac{d_n}{d_{n-2}} \left(\frac{n-1}{n} \right) = \gamma_n^2$

$\frac{d_n}{d_{n-1}} = \frac{n}{2n-1} \Rightarrow d_n = \frac{d_{n-1} \cdot n}{2n-1}$

COMPARISON : $\gamma_n^2 = \frac{d_n}{d_{n-2}} \left(\frac{n-1}{n} \right) =$
 $= \frac{d_{n-1}}{d_{n-2}} \left(\frac{1}{2n-1} \right) \left(\frac{n-1}{1} \right) = \left(\frac{n-1}{2(n-1)-1} \right) \left(\frac{n-1}{2n-1} \right) =$
 $= \frac{(n-1)^2}{(2n-3)(2n-1)} = \frac{(n-1)^2}{4(n-1)^2 - 1}$

(b)

```
In [13]: import numpy as np

#first we need to compute the gamma_k:
def gamma(k):
    if k == 1:
        return 0
    return (k - 1) / np.sqrt(4 * (k - 1)**2 - 1)

def get_gaussian_nodes_and_weights(n):
    main_diag = np.zeros(n) #elements in the diagonal AKA delta(n) = 0
    off_diag = np.array([gamma(k) for k in range(2, n+1)]) #The elements beside the diagonal AKA gamma(n). We are using gamma2, gamma3, ..., gamma(n)

    Jn = np.diag(main_diag) + np.diag(off_diag, k=1) + np.diag(off_diag, k=-1) # this is how the np combines the diagonal matrices

    #getting eigenvalues and eigenvectors:
    eigen_values, eigen_vectors = np.linalg.eig(Jn) #so now eigen_values = nodes

    #calculating the weights:
    v1 = eigen_vectors[0, :] # first row contains first components of each eigenvector
    lenght = np.sum(v1 ** 2) #we need to normalize the vectors

    weights = 2 * v1 ** 2 / lenght #calculating the weights by the formula

    calculated_nodes_and_weights = list(zip(eigen_values, weights)) #combine the nodes and their weights

    #built in function check:
    generated_nodes_and_weights = np.polynomial.legendre.leggauss(n)
    return calculated_nodes_and_weights, generated_nodes_and_weights

print(get_gaussian_nodes_and_weights(4))

([(-0.8611363115940525, 0.347854845137454), (-0.33998104358485615, 0.6521451548625466), (0.8611363115940531, 0.34785484513745357), (0.3399810435848563, 0.6521451548625457)], (array([-0.86113631, -0.33998104, 0.33998104, 0.86113631]), array([0.34785485, 0.65214515, 0.65214515, 0.34785485])))
```

(c)

```
In [25]: n = 4

#first we translate f like we learnt in the Lecture:
def g(t):
    return np.sin(np.pi / 2 * (1 + t)) #this was done by hand on paper

#now let's calculate the QR:
J_tilda = 0
nodes_with_weights_lst = get_gaussian_nodes_and_weights(n)[0]
for i in range(n):
    ci, bi = nodes_with_weights_lst[i]
    J_tilda += bi * g(ci)

J_tilda

1.2732295042595076
```

Comment: The absolute value is approximately 0.237 when n = 4